

1. Analyse de l'existant

Le projet initial se compose d'une API Java basée sur Spring Boot et d'une API Python utilisant FastAPI pour la recommandation. L'API Spring expose les entités fondamentales (jeu, auteur, éditeur, utilisateur, commandes) avec des contrôleurs rudimentaires. L'API Python intègre un unique *endpoint* fonctionnel, bien que l'algorithme de recommandation soit encore fictif.

Points forts de l'existant :

- Une structure Spring Boot classique, organisée en *controller*, *model*, *resources*, et assortie de tests initiaux.
- La présence de dépendances essentielles (Spring Web, Spring Data JPA, MySQL, outils de test).
- Une base FastAPI fonctionnelle avec des *endpoints* simples et des modèles Pydantic définis.

Lacunes principales :

- Une architecture incomplète : absence des couches *service*, des DTOs (*Data Transfer Objects*) et d'un mécanisme de mapping approprié.
- L'implémentation de la sécurité est inexistante (gestion des rôles client/administrateur non prise en charge).
- Des tests très limités, entraînant une couverture de code insuffisante.
- L'algorithme de recommandation par *Machine Learning* n'est pas encore fonctionnel (utilisation de données fictives).
- L'absence de documentation technique et de diagrammes d'architecture.

Audit de l'API Spring :

- **Évaluation critique** : Les entités sont présentes, mais le principe de séparation des responsabilités est ignoré, la logique étant trop souvent directement implémentée dans les contrôleurs. Les relations métiers (commandes, lignes, stocks, avis) existent, mais sans validation ni règles métiers explicites. La couche d'accès aux données, initiée via JPA, manque de *repositories* dédiés et d'une stratégie de requêtes claire.
- **Points faibles détaillés** :
 - *Modèles* : Manque de validation métier, d'annotations de sérialisation et de contraintes explicites.
 - *Contrôleurs* : Ils intègrent potentiellement de la logique métier et les *endpoints* sont incomplets.

- *Services* : Leur absence entrave la testabilité et le respect des principes SOLID.
- *DTOs* : Non utilisés, exposant directement les entités JPA.
- *Accès aux données* : Absence de *repositories* explicites et de méthodes de recherche dédiées.

2. Cadrage du projet

Le projet nécessite une refonte complète de l'API Spring vers une architecture REST solide et maintenable respectant les principes SOLID..

- **Exigences techniques :**
 - Mise en place de Hibernate/JPA avec des modèles cohérents et l'utilisation de DTOs.
 - Sécurisation via Spring Security (authentification/autorisation) avec gestion des rôles.
 - Implémentation d'un algorithme de recommandation KNN dans l'API Python.
 - Mise en place de la communication entre l'API Spring et l'API Python pour récupérer les recommandations.
 - Production d'une documentation technique exhaustive incluant diagrammes UML et synthèse des bonnes pratiques.
- **Exigences fonctionnelles :**
 - Implémentation complète du CRUD (*Create, Read, Update, Delete*) pour les entités principales (jeux, utilisateurs, commandes, auteurs, éditeurs, catégories, inventaires, avis, listes de souhaits).
 - Gestion des rôles CLIENT et ADMIN avec une définition claire des droits associés.
 - Fonctionnalités de recherche avancée sur les jeux (par titre, auteur, catégorie, éditeur).
 - Intégration d'un *endpoint* pour les recommandations personnalisées basées sur les données utilisateur.
- **Exigences non-fonctionnelles :**
 - Respect strict des principes SOLID et d'une architecture REST claire et cohérente.
 - Mise en œuvre de Spring Security pour l'authentification et l'autorisation.
 - Atteindre une couverture de tests (unitaires et d'intégration) d'au moins 70 %.

3. Choix de conception

Les endpoints suivants gèreront les données fondamentales de l'application :

- `/api/games` : Catalogue de jeux.
- `/api/users` : Gestion des comptes utilisateurs.
- `/api/users/{userId}/recommendations` : Intégration des résultats du moteur de recommandation pour un utilisateur donné..
- `/api/orders` : Processus de commande et détails des lignes de commande.
- `/api/authors`, `/api/publishers`, `/api/categories` : Services de référentiels.
- `/api/wishlists` et `/api/reviews` : Fonctionnalités sociales et d'interaction utilisateur.

Séparation des Préoccupations (Clean Architecture)

L'application adoptera une structure en couches pour garantir la maintenabilité et la testabilité :

- **Controller** : Gère la couche web. Responsable de la validation des requêtes HTTP, du mapping des DTOs d'entrée et de sortie, et de l'attribution des codes de réponse HTTP.
- **Service** : Contient la logique métier centrale. Il applique les règles de validation métier et orchestre les interactions avec les systèmes externes (Repositories, autres services).
- **Repository** : Couche d'accès aux données. Utilisation de Spring Data JPA pour les opérations de persistance.

Stratégie DTOs et Mappers

- **DTOs dédiés** : Utilisation de DTOs (Data Transfer Objects) distincts pour les opérations d'entrée (*Request) et de sortie (*Response), par exemple `GameCreateRequest` et `GameResponse`.
- **Isolation des Entités** : Les entités JPA (modèles de données) ne seront jamais exposées directement par l'API afin de découpler la couche de persistance de la couche API.
- **Gestion des Relations** : Les relations seront incluses de manière sélective dans les DTOs de sortie pour prévenir les boucles de référence et éviter la surcharge des payloads JSON.

Cette architecture a été choisie afin de garantir la maintenabilité, la testabilité et l'évolutivité de l'application. La séparation claire entre les couches Controller, Service et Repository permet de respecter les principes SOLID et de limiter le couplage entre les différentes parties du système.

4. Travail sur l'API de recommandations

Dans le cadre de l'amélioration de la plateforme, un système de recommandation a été mis en place afin de proposer des jeux pertinents aux utilisateurs en fonction de leurs préférences et de leur historique d'achats. Ce système repose sur une API développée en Python avec FastAPI, permettant d'intégrer un algorithme de Machine Learning tout en conservant une séparation claire entre la logique métier de l'application et le moteur de recommandation.

L'API Python agit comme un micro-service dédié à l'analyse des données utilisateurs et à la génération de recommandations. Elle expose deux routes principales : une route de vérification du service et un endpoint dédié à la génération de recommandations. Les requêtes sont validées à l'aide de modèles Pydantic, ce qui garantit la cohérence des données reçues.

Les données nécessaires au calcul des recommandations sont récupérées depuis la base MySQL GamesUP à l'aide de SQLAlchemy. Les avis et notes des utilisateurs sont transformés en DataFrame, puis organisés sous forme d'une matrice utilisateur-jeu représentant les préférences des utilisateurs.

Le moteur de recommandation repose sur l'algorithme K-Nearest Neighbors (KNN). Cet algorithme permet d'identifier les utilisateurs ayant des comportements similaires en calculant la similarité cosinus entre leurs profils. Les utilisateurs les plus proches sont ensuite utilisés pour déterminer les jeux susceptibles d'intéresser l'utilisateur cible.

Un mécanisme de fallback est également prévu afin de garantir une réponse dans tous les cas. Lorsqu'aucune similarité significative n'est trouvée ou lorsqu'un utilisateur ne possède pas d'historique d'achats, le système renvoie une liste des jeux les mieux notés globalement.

L'intégration avec l'API Spring Boot se fait via un appel HTTP vers le service FastAPI. L'API Spring transmet les données utilisateur, puis récupère la liste des recommandations générées afin de les renvoyer au frontend. Cette architecture permet de découpler complètement le moteur de recommandation du reste de l'application.

Bonnes pratiques appliquées:

Plusieurs bonnes pratiques ont été appliquées lors du développement du service :

- **Séparation des responsabilités** entre les modules de chargement des données, les modèles et le moteur de recommandation.
- **Utilisation de variables d'environnement** pour la configuration de la base de données.
- **Validation stricte** des requêtes grâce aux modèles Pydantic.
- **Mise en place d'un cache mémoire** pour limiter les accès répétés à la base de données et améliorer les performances.

Le choix d'un micro-service Python dédié au système de recommandation permet également d'améliorer la scalabilité de l'application. Le moteur de recommandation peut évoluer indépendamment de l'API principale et être amélioré ou remplacé sans impacter le reste du système.

5. Documentations

Les diagrammes suivants présentent l'architecture du système, le modèle de données ainsi que les principaux flux applicatifs.

La refonte de cette API a permis de transformer une base de code initialement peu structurée en une architecture plus robuste et maintenable. Ce travail met en évidence l'importance d'une bonne conception logicielle dès les premières phases d'un projet afin de limiter la dette technique et de faciliter les évolutions futures.

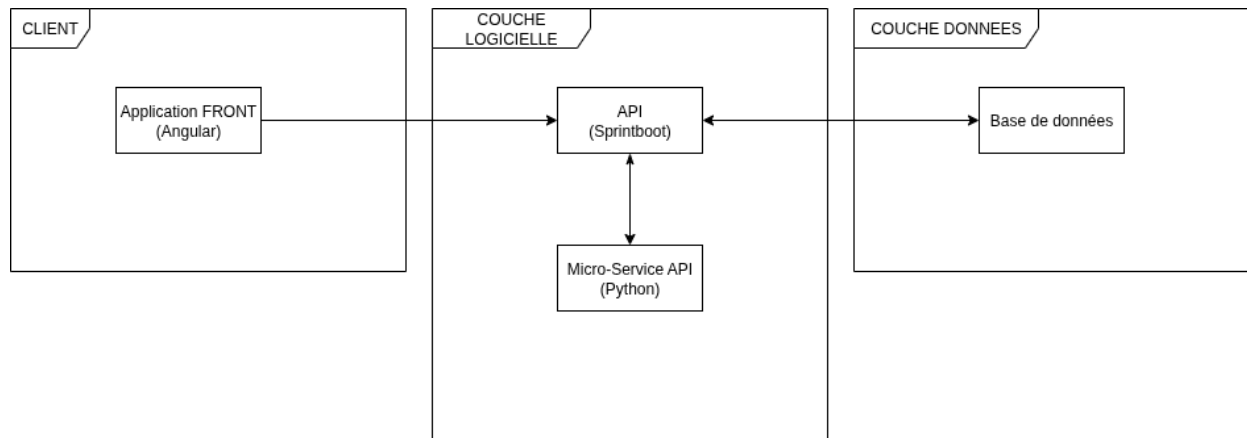


Figure 1 : Diagramme d'achitecture

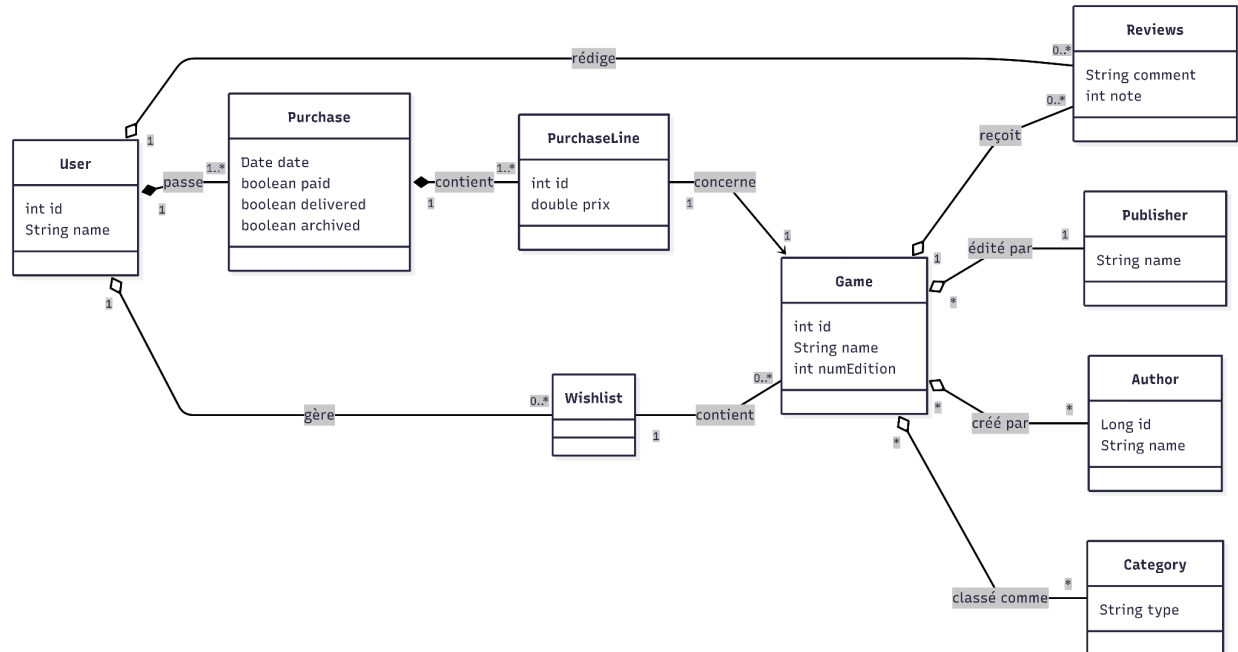


Figure 2 : Diagramme de classe

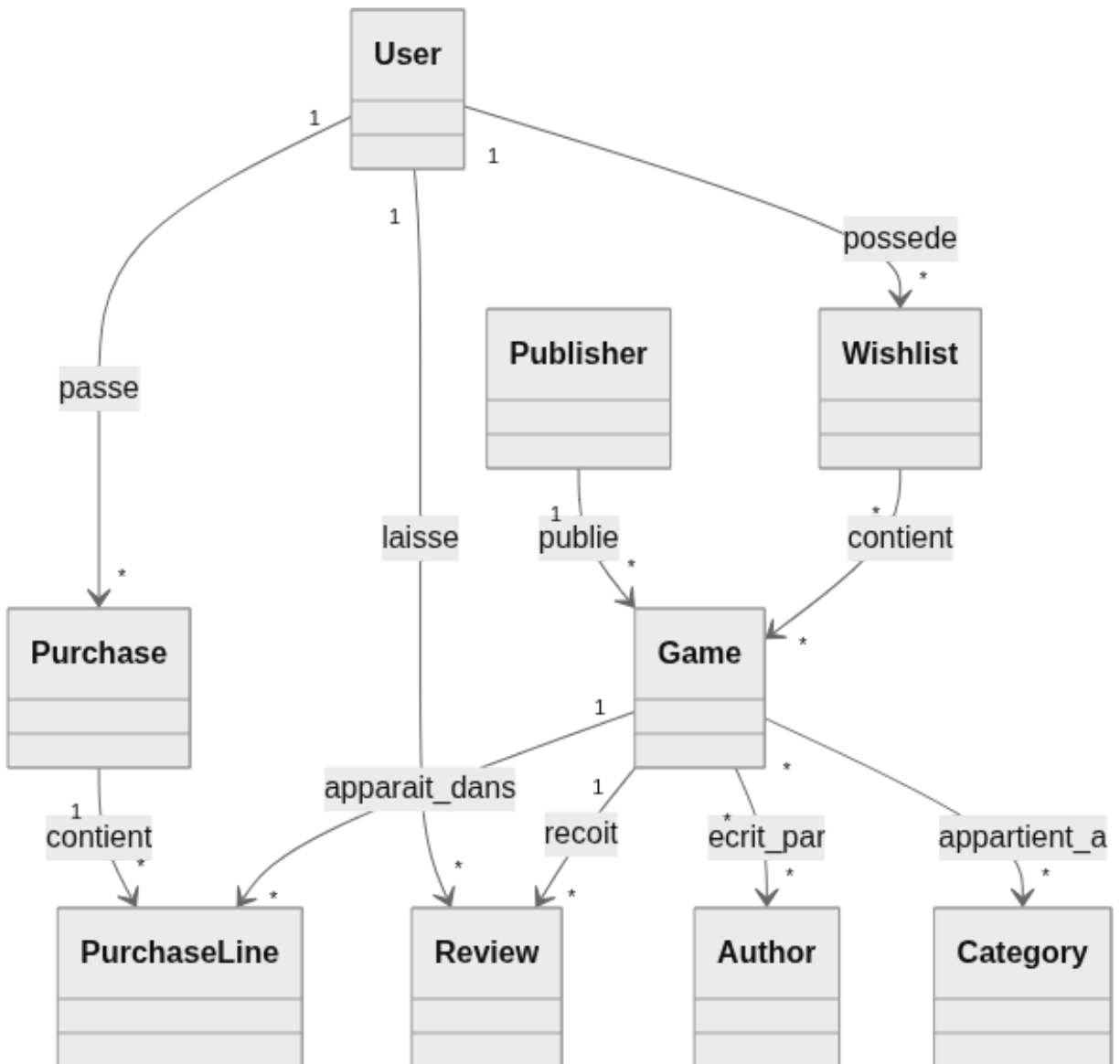


Figure 3 : Modèle conceptuel de base de donnée

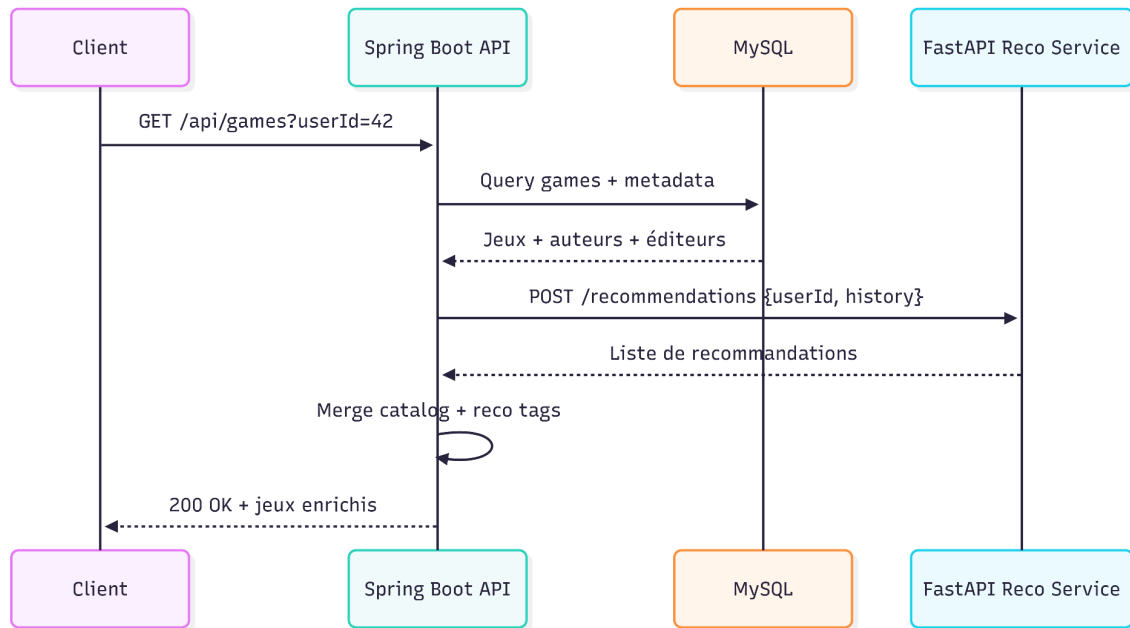


Figure 4 : Diagramme de séquence (Recherche Jeux)

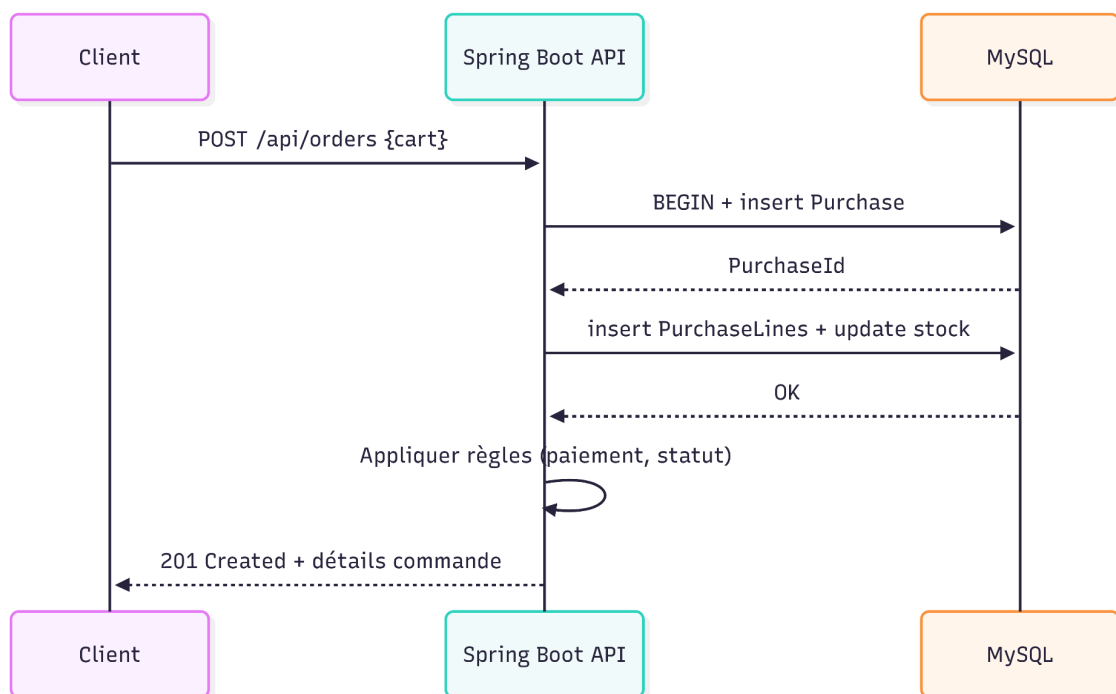


Figure 5 : Diagramme de séquence (Commande)

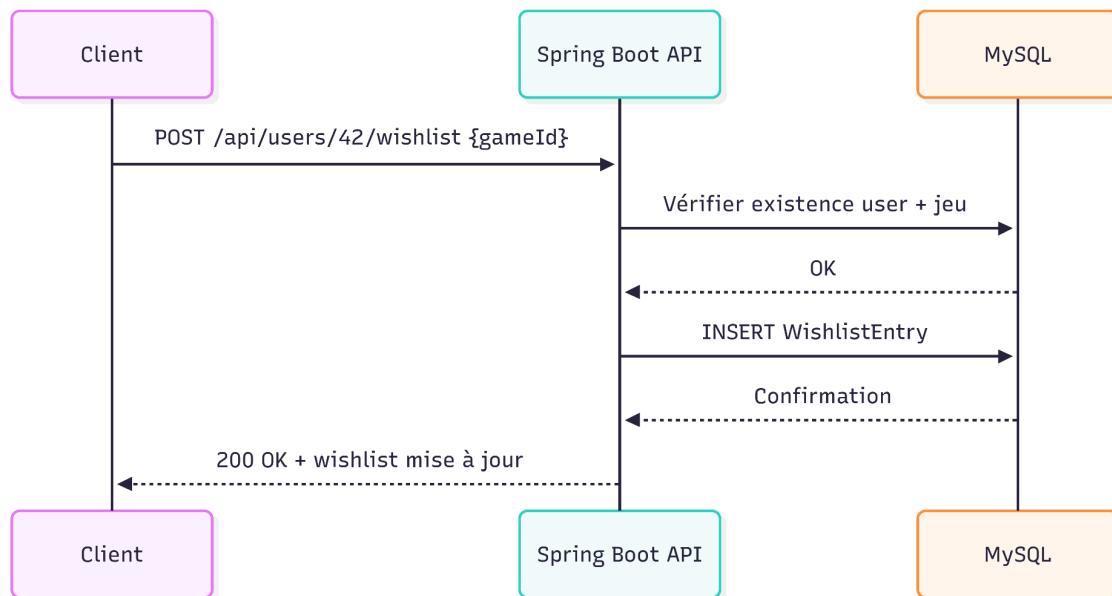


Figure 6 : Diagramme de séquence (Liste de souhait)

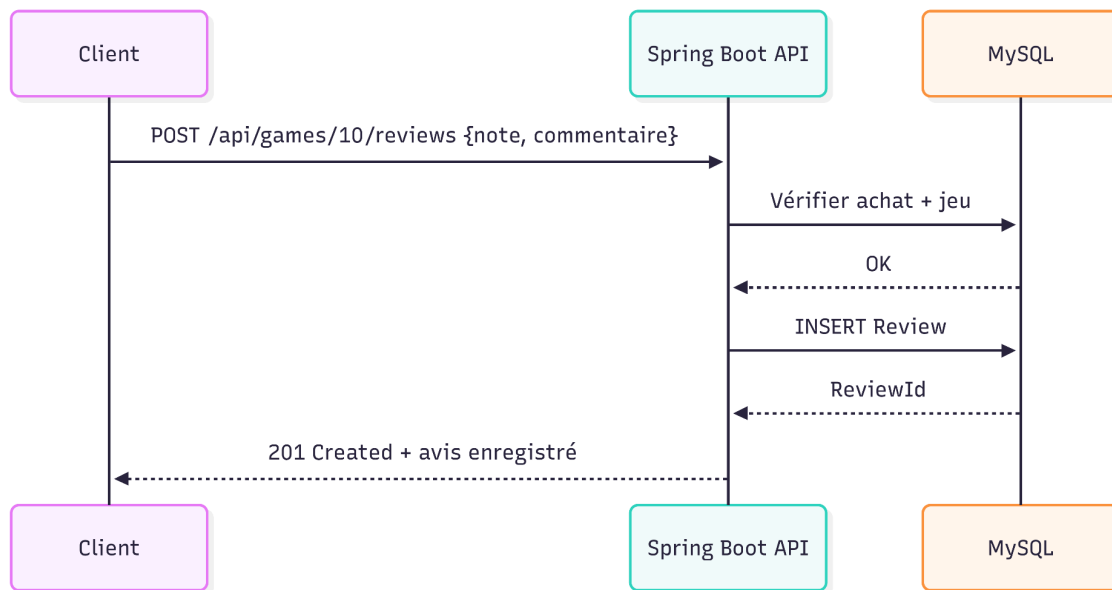


Figure 7 : Diagramme de séquence (Review)

gamesUP

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.gamesUP.gamesUP.controller		40 %		37 %	32 52	57 105	26 44	0 9
com.gamesUP.gamesUP.security		25 %		18 %	16 20	43 60	5 9	0 3
com.gamesUP.gamesUP.recommendation.service		17 %		0 %	9 12	31 43	6 9	0 2
com.gamesUP.gamesUP.model		71 %		n/a	37 127	54 188	37 127	0 11
com.gamesUP.gamesUP.recommendation.dto		0 %		0 %	5 5	6 6	4 4	4 4
com.gamesUP.gamesUP.dto		88 %		n/a	3 62	3 90	3 62	3 6
com.gamesUP.gamesUP.recommendation.controller		15 %		0 %	2 3	4 5	1 2	0 1
com.gamesUP.gamesUP.service		98 %		87 %	11 102	1 207	1 61	0 9
com.gamesUP.gamesUP.recommendation.exception		0 %		n/a	2 2	4 4	2 2	1 1
com.gamesUP.gamesUP.recommendation.config		78 %		50 %	2 5	1 6	0 3	0 2
com.gamesUP.gamesUP		37 %		n/a	1 2	2 3	1 2	0 1
com.gamesUP.gamesUP.repository.specification		100 %		96 %	1 16	0 34	0 2	0 1
com.gamesUP.gamesUP.config		100 %		n/a	0 9	0 31	0 9	0 2
Total	792 of 2 924	72 %	51 of 162	68 %	121 417	206 782	86 336	8 52

Figure 8: Rapport couverture de code

La mise en place de tests unitaires et d'intégration permet de garantir la stabilité de l'application et de prévenir les régressions lors des évolutions du code.

6. Code source

Le code source est versionné sur mon dépôt GitHub.

Vous trouverez le repos a l'adresse suivante: <https://github.com/WINK3R/gameup-exercice>

Pour toutes questions ou debug je suis disponible à l'adresse delanierlucas@gmail.com